DOCUMENTO DE DISEÑO DEL SISTEMA

Platzi Wallet

Billetera virtual cloud-ready con enfoque profesional, factible y progresivo

Versión	1.0
Estado	Documento académico-técnico
Fecha	Abril 2026
Proyecto	Platzi Wallet
Arquitectura base	Monolito modular cloud-ready
Stack inicial sugerido	Laravel, PostgreSQL/MySQL, Redis, AWS

1. Resumen ejecutivo

Platzi Wallet se plantea como una billetera digital orientada a pagos entre personas, pagos a terceros, operaciones mediante códigos QR, consignaciones y retiros de fondos. El sistema debe responder a necesidades reales de disponibilidad, trazabilidad y orden funcional, pero sin caer desde el inicio en una arquitectura excesivamente compleja para el alcance del proyecto.

Por esa razón, la propuesta adopta un enfoque de monolito modular cloud-ready. Esta decisión permite mantener una base central bien estructurada, separar dominios de negocio dentro del mismo backend y dejar abierta la posibilidad de evolucionar hacia microservicios cuando el crecimiento del sistema realmente lo exija.

2. Planteamiento del sistema

2.1 Descripción general

Platzi Wallet es una aplicación financiera digital que centraliza operaciones de dinero electrónico. El usuario podrá autenticarse, consultar saldo, revisar movimientos, transferir dinero a otros usuarios, pagar a terceros, operar con códigos QR y recibir notificaciones de sus transacciones. La primera aproximación del sistema se orienta a una versión web responsiva o PWA, dejando la aplicación móvil nativa para etapas posteriores.

2.2 Objetivo general

Diseñar una plataforma de pagos digitales sobre la nube que sea profesional, implementable y mantenible, con una arquitectura suficientemente sólida para crecer por etapas sin reescrituras prematuras.

2.3 Objetivos específicos

1. Definir una base funcional para pagos P2P, pagos a terceros, QR, consignaciones y retiros.
2. Establecer una arquitectura ordenada que relacione cliente, backend, persistencia e integraciones externas.
3. Justificar la adopción de servicios cloud y un stack tecnológico realista para una implementación posterior.

4. Dejar preparada la solución para profundizar seguridad, infraestructura y evolución del sistema en entregas siguientes.

3. Alcance, supuestos y límites

3.1 Alcance funcional considerado

- Registro e inicio de sesión de usuarios.
- Consulta de saldo e historial de movimientos.
- Pagos P2P entre usuarios registrados.
- Pagos a terceros mediante integraciones externas.
- Generación y escaneo de códigos QR.
- Consignaciones y retiros de fondos.
- Notificaciones de eventos relevantes para el usuario.

3.2 Supuestos

- Los usuarios acceden a la plataforma desde dispositivos con conectividad a Internet.
- Se dispone de una pasarela de pagos que expone APIs utilizables por el sistema.
- La nube objetivo inicial será AWS por madurez de servicios y afinidad con el stack propuesto.
- El proyecto se construirá de forma progresiva, priorizando primero un núcleo funcional estable.

3.3 Fuera de alcance en esta etapa

- Criptomonedas, inversiones, créditos y productos financieros avanzados.
- Procesamiento internacional con múltiples divisas en la primera versión.
- Panel regulatorio o contable avanzado.
- Aplicación móvil nativa completa desde el inicio.

4. Justificación técnica de la propuesta

La nube resulta adecuada para este tipo de sistema porque combina elasticidad, disponibilidad y acceso a servicios gestionados. En una billetera digital, la carga de transacciones no es constante y puede intensificarse en horas pico, campañas comerciales o periodos de pago. Una plataforma cloud permite absorber esas variaciones sin depender de infraestructura física propia ni de operaciones manuales extensas.

- Reduce la fricción operativa al apoyarse en servicios gestionados para base de datos, colas, almacenamiento y monitoreo.
- Facilita el crecimiento del sistema mediante escalado progresivo.
- Permite construir una base preparada para observabilidad, auditoría y seguridad avanzada.
- Hace viable una posterior automatización con CI/CD y despliegue controlado.

El uso de Laravel como base del backend también se justifica por factibilidad. El framework ofrece una estructura madura para autenticación, APIs, validaciones, colas, eventos, notificaciones y acceso a base de datos. Esto lo convierte en una opción muy sólida para construir una primera versión seria sin recurrir desde el primer momento a un ecosistema distribuido más costoso de operar.

5. Decisión arquitectónica

5.1 Comparación de enfoques

Se analizaron tres enfoques: monolito tradicional, monolito modular y microservicios. El monolito tradicional es simple, pero tiende a desordenarse con rapidez. Los microservicios ofrecen mayor desacoplamiento y escalado independiente, pero exigen más madurez operativa, observabilidad, coordinación entre servicios y esfuerzo de desarrollo. El monolito modular se ubica en un punto intermedio y representa el mejor equilibrio para este proyecto.

Enfoque	Fortaleza	Riesgo principal	Factibilidad para el proyecto
Monolito	Implementación rápida y simple.	Se vuelve rígido si crece sin orden.	Media
Monolito modular	Orden interno, mantenibilidad y crecimiento progresivo.	Escalado menos fino que microservicios.	Alta
Microservicios	Desacoplamiento y escalado independiente.	Complejidad operativa temprana.	Media/Baja para la primera etapa

5.2 Enfoque seleccionado

Se selecciona una arquitectura de monolito modular cloud-ready. Esta propuesta mantiene un solo backend principal, pero organiza el dominio en módulos claramente identificables: autenticación, wallet, pagos, QR, consignaciones, retiros y notificaciones. La decisión es coherente con la meta del documento: construir un sistema profesional y real, no una maqueta sobrecargada de componentes difíciles de implementar.

6. Vistas arquitectónicas

6.1 Diagrama a nivel del sistema

Esta vista muestra a Platzi Wallet como un sistema central que interactúa con usuarios y servicios externos. Su objetivo es aclarar el contexto general del sistema y los límites de integración.

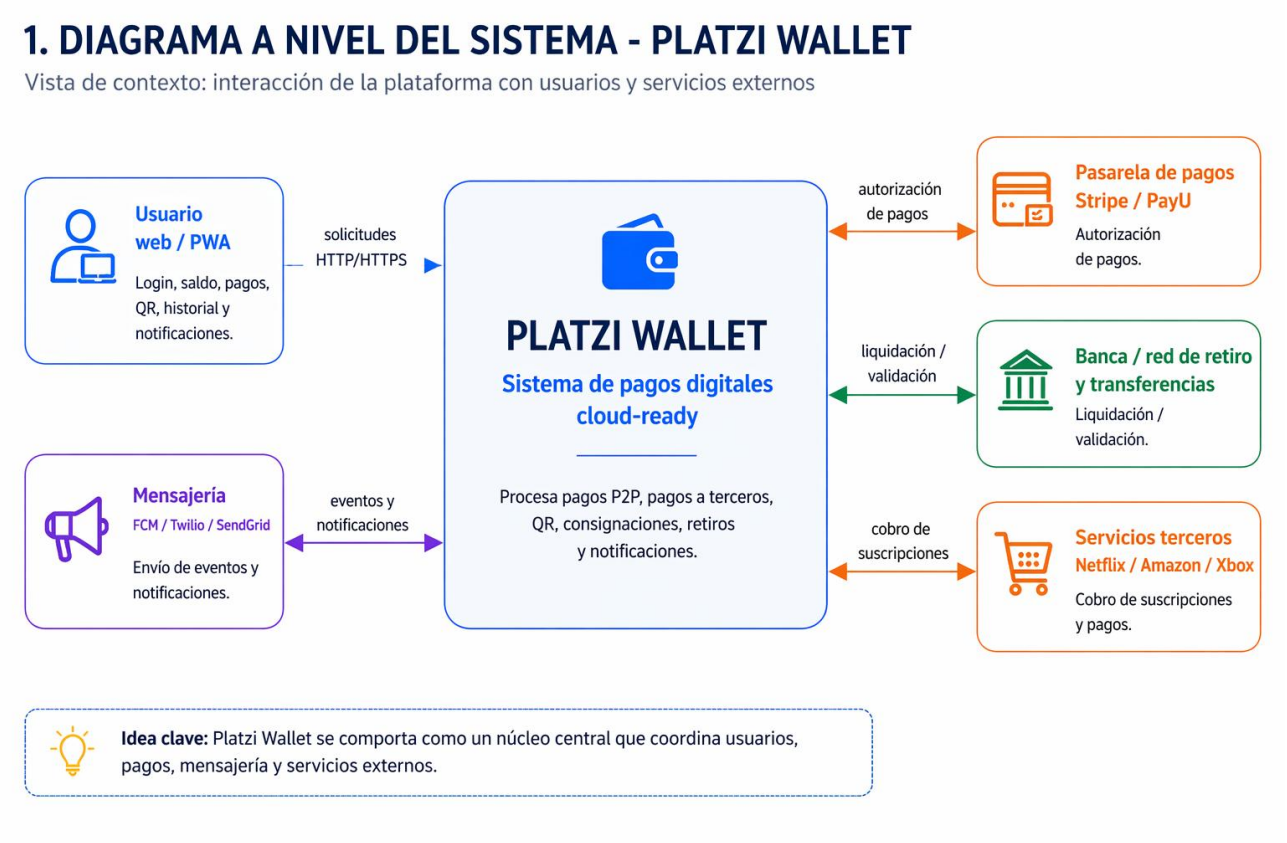

Figura 1. Vista de contexto del sistema Platzi Wallet.

6.2 Diagrama de arquitectura del sistema

La arquitectura interna se organiza por capas: cliente, distribución del frontend, entrada segura, backend principal, persistencia y servicios transversales. Las integraciones externas se relacionan con el backend y no con el usuario de forma directa, lo cual ayuda a mantener control y trazabilidad.

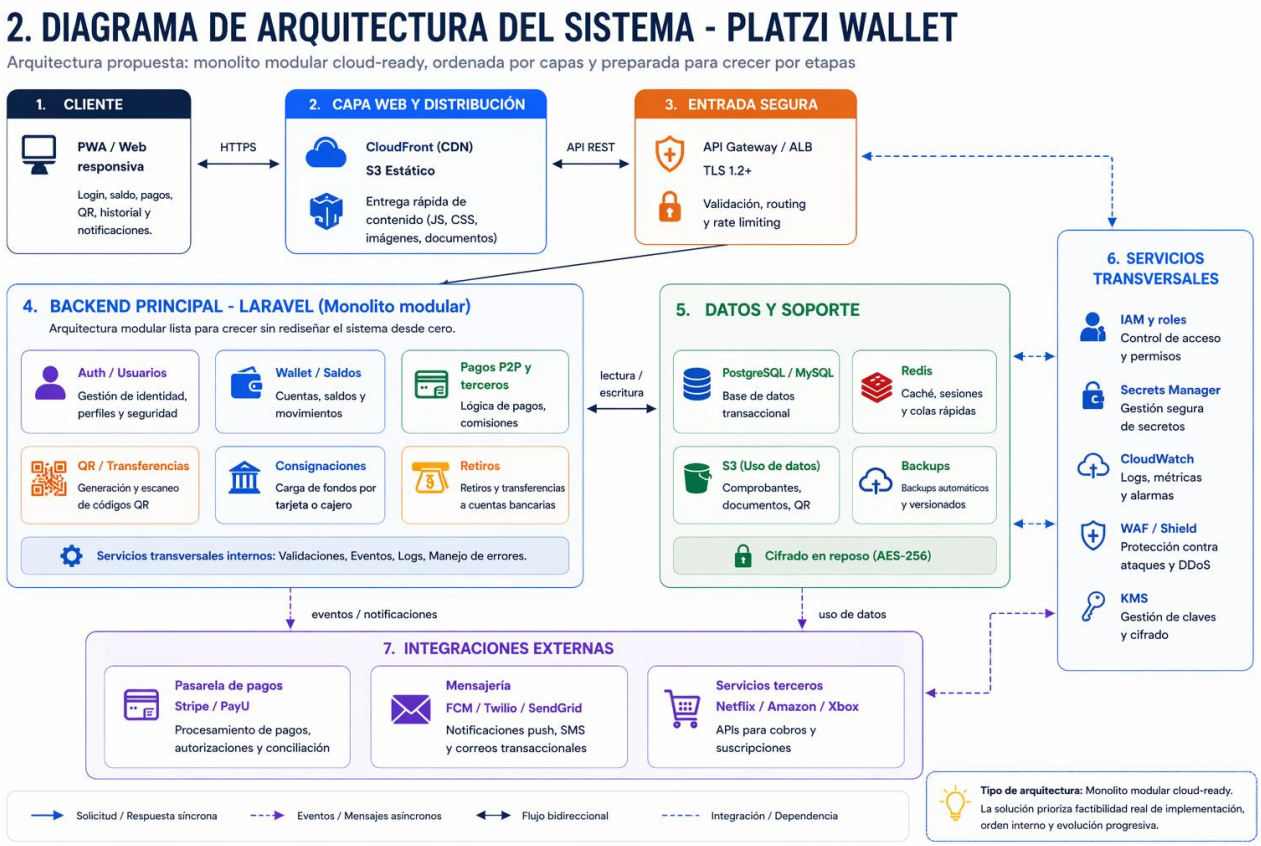

Figura 2. Arquitectura propuesta del sistema.

6.3 Diagrama de secuencia de flujo de extremo a extremo

El siguiente flujo representa la operación principal de pago P2P. Esta vista permite verificar cómo interactúan el usuario, la entrada segura, el backend, la persistencia y los servicios de notificación dentro de una transacción completa.

Figura 3. Flujo extremo a extremo del pago P2P.

7. Explicación detallada de la arquitectura

7.1 Cliente

La capa cliente corresponde a la interfaz con la que opera el usuario final. Desde aquí se gestionan inicio de sesión, consulta de saldo, generación y escaneo de QR, pagos y consulta de historial. Se recomienda iniciar con una web responsiva o PWA para mantener factibilidad sin cerrar la puerta a una futura app móvil.

7.2 Capa web y distribución

El frontend y los activos estáticos pueden distribuirse mediante S3 y CloudFront. Esta decisión desacopla la entrega del frontend del backend principal, mejora tiempos de carga y prepara la solución para un manejo más eficiente de archivos, imágenes, hojas de estilo y scripts.

7.3 Entrada segura

La entrada al sistema se concentra en un punto controlado, ya sea mediante API Gateway o un Application Load Balancer. Aquí se unifican HTTPS, validación básica de solicitudes, routing y políticas iniciales de control de tráfico. Esto ordena el acceso al backend y reduce la exposición directa de la aplicación.

7.4 Backend principal - Laravel

El backend central implementa la lógica del negocio. En lugar de distribuir responsabilidades en servicios independientes desde la primera versión, se propone organizar Laravel por módulos. Cada módulo agrupa entidades, casos de uso, validaciones y eventos propios, manteniendo cohesión funcional sin perder simplicidad de despliegue.

7.5 Datos y soporte

La persistencia principal se apoya en PostgreSQL o MySQL para usuarios, billeteras, transacciones e historial. Redis cumple dos roles clave: cachear información de consulta frecuente y actuar como soporte para colas rápidas. S3 sirve para almacenar QR, comprobantes y otros objetos complementarios. Backups y versionado completan la base de continuidad operativa.

7.6 Integraciones externas y servicios transversales

El sistema requiere integración con pasarelas de pago, servicios de mensajería y proveedores externos. Paralelamente, necesita capacidades transversales de IAM, secretos, monitoreo, cifrado y protección perimetral. Estas funciones no son módulos de negocio, pero atraviesan toda la arquitectura y condicionan su solidez.

8. Componentes principales

Componente	Responsabilidad principal	Justificación
Cliente web/PWA	Interacción del usuario con la plataforma.	Permite una primera implementación realista y usable.
CloudFront + S3	Distribución de frontend y activos.	Desacopla contenido estático y mejora entrega.
Entrada segura	Unifica acceso y routing hacia el backend.	Ordena exposición del sistema y prepara seguridad.
Laravel modular	Implementa la lógica del negocio.	Equilibra profesionalismo y factibilidad.
PostgreSQL / MySQL	Persistencia transaccional.	Motores maduros y adecuados para el dominio.
Redis	Cache y colas rápidas.	Mejora rendimiento y apoya asincronía.
S3	Almacenamiento de QR y comprobantes.	Simplifica manejo de archivos y escalado.
Servicios transversales	IAM, secretos, monitoreo, cifrado.	Preparan el crecimiento seguro del sistema.

9. Flujo principal del negocio: pago P2P

5. El usuario emisor inicia sesión y registra la intención de transferir dinero.
6. La capa de entrada segura valida el acceso y reenvía la solicitud al backend.
7. Laravel valida reglas básicas, saldo disponible y estado de las cuentas involucradas.
8. La base de datos registra una transacción idempotente y actualiza saldos en una operación consistente.
9. Se almacena el movimiento en el historial y se emite un evento para notificación.
10. El usuario emisor recibe confirmación y el usuario receptor es notificado del abono.

Este flujo concentra los aspectos más críticos del sistema: autenticación, consistencia transaccional, trazabilidad y comunicación. Por esa razón se usa como escenario principal para describir la arquitectura.

10. Propuesta tecnológica inicial

Capa	Tecnología sugerida	Motivo de selección
Frontend	Web responsiva / PWA	Reduce complejidad inicial y mantiene buena experiencia de usuario.
Backend	Laravel	Framework maduro, ordenado y productivo para construir el núcleo.
Base de datos	PostgreSQL o MySQL	Motores ampliamente soportados y adecuados para transacciones.
Cache / colas	Redis	Acelera consultas y habilita procesamiento asíncrono.
Activos estáticos	S3 + CloudFront	Entrega eficiente del frontend y archivos asociados.
Entrada	API Gateway o ALB	Controla acceso, routing y futura seguridad perimetral.
Nube objetivo	AWS	Madurez de servicios y mejor alineación con el enfoque del proyecto.

11. Consideraciones de factibilidad

La factibilidad del sistema no depende solo de que el diagrama se vea bien. Depende de que las decisiones técnicas puedan sostenerse con el equipo, el tiempo y el nivel de implementación esperado. Por ello, la propuesta evita separar prematuramente servicios que aún pueden convivir dentro de un backend modular.

- Permite avanzar por etapas sin reescribir la base del sistema.
- Facilita una eventual implementación real con Laravel y servicios cloud gestionados.
- Reduce riesgo técnico frente a una propuesta de microservicios completa desde el inicio.
- Mantiene un nivel profesional sin caer en sobreingeniería temprana.

12. Preparación para la evolución futura

La arquitectura propuesta no se cierra a crecer. Al contrario, deja una ruta clara: profundizar infraestructura cloud, separar componentes si la carga lo exige e incorporar seguridad avanzada en una entrega posterior. Esto significa que la propuesta es progresiva, no improvisada. Primero se define bien el sistema; después se madura la operación.

Bajo este enfoque, módulos como pagos, notificaciones o QR podrían aislarse como servicios independientes en el futuro. Sin embargo, esa evolución deberá responder a necesidad real de escalado, equipo o dominio, no a un deseo prematuro de fragmentación arquitectónica.

13. Conclusión

La propuesta ordenada para Platzi Wallet se apoya en una arquitectura de monolito modular cloud-ready porque ofrece el mejor equilibrio entre profesionalismo, claridad y posibilidad real de ejecución. El

sistema queda suficientemente estructurado para explicar su funcionamiento, justificar sus componentes y sentar una base seria de construcción progresiva.

En síntesis, el documento no busca inflar la solución con complejidad innecesaria, sino establecer una arquitectura coherente, entendible y técnicamente defendible. Esa es justamente la base que necesita un proyecto que quiere construirse poco a poco sin perder dirección.